Relatório e Documentação da API: iScrev XBoard

> ***Projeto:*** *iScrev XBoard - A Lousa Digital do Renascimento Moderno* ***Classificação:*** *PWA (Progressive Web App) + Desktop App (Electron)* ***Objetivo:*** *Fornecer um resumo arquitetural, estrutural e detalhes de implementação da API do sistema para referência futura e manutenção.*

---

1. Visão Geral da Arquitetura

O **iScrev XBoard** é uma lousa digital de alta performance desenvolvida para aulas e reuniões, encapsulada tanto para a Web (PWA offline via *Stale-While-Revalidate* e `sw.js`) quanto para Desktop via **Electron**.

A arquitetura JavaScript no Frontend abandona o antigo estilo monolítico ES5 (visto originalmente no iScrev Notes) em favor de uma organização modular usando **Classes ES6** divididas em pastas lógicas: `core/`, `modules/`, `services/`, e `libs/`. A injeção de dependências é resolvida atrelando as instâncias principais ao objeto global `window` durante o evento `load`.

Estrutura do Electron (`main.js` e `preload.js`)

- **Isolamento de Contexto:** Usa `contextIsolation: true` e `nodeIntegration: false`. A comunicação entre a tela (Renderer) e o SO (Main) se dá puramente pelo `preload.js` usando `contextBridge`.
- **Estúdio de Gravação Nativo:** A interceptação de mídia no Electron é gerenciada por `session.defaultSession.setDisplayMediaRequestHandler` acoplado ao `desktopCapturer`. A aplicação é capaz de gravar a tela principal fundindo áudio de loopback e de microfone (captura nativa do sistema).
- **Webviews Habilitadas:** A tag `<webview>` é permitida por razões de isolamento (como rodar iframes do YouTube de forma segura).

---

2. API e Componentes Centrais (Frontend)

O motor do XBoard roda em torno de um conjunto de Classes Singleton que se comunicam entre si. Abaixo, detalhamos os componentes cruciais do sistema:

2.1. `CanvasEngine` (Engine Gráfica)

Responsável por interpretar eventos do mouse/caneta e traduzi-los em curvas perfeitamente suaves na tag `<canvas>` com `desynchronized: true` (para baixar a latência).

- **Estado:**
  - `isDrawing`: boolean controlando o pointer.
  - `currentTool`: Ferramenta ativa (`pen`, `eraser`).
  - `currentStroke`: Objeto temporário com os pontos sendo desenhados.
  - `currentColor`, `currentLineWidth`, `lastPressure`: Configurações visuais.
- **Funções Principais:**
  - `onPointerDown`, `onPointerMove`, `onPointerUp`: Gerenciam a entrada via Pointer Events.
  - O `onPointerMove` gera uma **Curva Quadrática de Bézier** (`quadraticCurveTo`), calculando dinamicamente o ponto médio entre o ponto atual e anterior para arredondar as "quinas" da escrita.
  - O *cap* e o *join* são `round` por padrão, e a espessura (`lineWidth`) pode variar dinamicamente pela pressão (`e.pressure`).

- `redrawHistory(strokes)` : Limpa o Canvas e reconstrói completamente todos os traçados fornecidos. Fundamental para paginação e *Undo/Redo*.

2.2. `History` / `HistoryManager` (Paginação e Desfazer)

Esta classe é o cérebro do estado da lousa. Ela gerencia o array de *Boards* (Pranchetas) e a máquina do tempo.

- **Estrutura de Dados (`Board`):**
  - O projeto funciona como um caderno de múltiplas páginas. Cada "página" (board) é representada por:

```
{
  "id": 1691234567,
  "strokes": [...],
  "redoStack": [...],
  "bg": { "type": "solid", "color": "#1e1e1e" }
}
```

- **Estado Global do Histórico:**
  - `boards` : Um array de configurações de lousas.
  - `currentIndex` : O índice apontando para a lousa atualmente visível na tela.
- **Funções Principais:**
  - `addStroke(stroke)` : Registra um novo traço, esvazia a pilha de `redoStack` e envia para persistência.
  - `undo()` , `redo()` : Manipulam as listas `strokes` e `redoStack` , e em seguida invocam o CanvasEngine para re-renderizar a tela `redraw()` .
  - `nextBoard()` , `prevBoard()` , `addBoard()` , `removeBoard()` : Funções de paginação. Mudar de página aciona internamente `changeBoard()` , que redesenha a tela inteira a partir do zero instantaneamente.
  - `saveToStorage()` : Executa um *Debounce* de 1 segundo (1000ms) após qualquer alteração para delegar os dados ao `StorageService` , prevenindo travamentos durante a escrita rápida.

2.3. `StorageService` (Persistência)

Abstrai a conexão com o `IndexedDB` nativo do navegador para uso off-line e rápido.

- **Banco e Tabela:** Usa o banco `XBoardDB` versão 1, operando no Store `sessions` .
- **Lousa Ativa (Sessão Corrente):**
  - `saveSession(sessionData)` : Sobrescreve o ID fixo `'current'` sempre que a lousa ativa é modificada (por debounce do HistoryManager).
  - `loadSession()` : Invocado automaticamente ao inicializar a página para resgatar o que estava na lousa antes de fechar.
- **Biblioteca de Aulas (Projetos salvos):**
  - `saveLessonAs(title)` : Pega os quadros (`boards`) atuais, gera um UUID (ex: `lesson_123456`) e persiste na biblioteca para carregamento futuro.
  - `getAllLessons()` , `loadLesson(id)` , `deleteLesson(id)` : Lógica de CRUD completa para a administração das aulas locais.

---

3. Detalhes de Integração e UI

A UI é gerenciada modularmente e interage com os "Cores". Para expansão futura:

- As atualizações de número de páginas na barra superior, botões e painéis são propagadas de volta via um `UIManager` (ex: `window.UIManager.updateBoardCounter`).
- A responsividade utiliza estilos em *Glassmorphism* no `css/` e um menu lateral estilo Gaveta (Drawer) para mobile.
- A exportação em PDF, se necessária, captura a saída visual rasterizada do Canvas atual ou dos Boards do `HistoryManager`.

4. Scripts e Automação de Build

Para facilitar o ecossistema PWA vs Electron, o projeto possui o `package.json` configurado:

- `npm run dev`: Ambiente iterativo.
- `npm run build`: Empacota o Windows usando `electron-builder` (Instalador Setup e Versão Portable encapsulada no `dist/`).
- `npm run build:web`: Processo estrito para a distribuição do PWA no servidor Web.